1.7 Exception Handling

Exceptions are unexpected events that occur during the execution of a program. An exception might result from a logical error or an unanticipated situation. In Python, ***exceptions*** (also known as ***errors***) are objects that are ***raised*** (or ***thrown***) by code that encounters an unexpected circumstance. The Python interpreter can also raise an exception should it encounter an unexpected condition, like running out of memory. A raised error may be ***caught*** by a surrounding context that "handles" the exception in an appropriate fashion. If uncaught, an exception causes the interpreter to stop executing the program and to report an appropriate message to the console. In this section, we examine the most common error types in Python, the mechanism for catching and handling errors that have been raised, and the syntax for raising errors from within user-defined blocks of code.

Common Exception Types

Python includes a rich hierarchy of exception classes that designate various categories of errors; Table 1.6 shows many of those classes. The Exception class serves as a base class for most other error types. An instance of the various subclasses encodes details about a problem that has occurred. Several of these errors may be raised in exceptional cases by behaviors introduced in this chapter. For example, use of an undefined identifier in an expression causes a NameError, and errant use of the dot notation, as in foo.bar( ), will generate an AttributeError if object foo does not support a member named bar.

Class	**Description**
Exception	A base class for most error types
AttributeError	Raised by syntax obj.foo, if obj has no member named foo
EOFError	Raised if "end of file" reached for console or file input
IOError	Raised upon failure of I/O operation (e.g., opening file)
IndexError	Raised if index to sequence is out of bounds
KeyError	Raised if nonexistent key requested for set or dictionary
KeyboardInterrupt	Raised if user types ctrl-C while program is executing
NameError	Raised if nonexistent identifier used
StopIteration	Raised by next(iterator) if no element; see Section 1.8
TypeError	Raised when wrong type of parameter is sent to a function
ValueError	Raised when parameter has invalid value (e.g., sqrt(−5))
ZeroDivisionError	Raised when any division operator used with 0 as divisor

Table 1.6: Common exception classes in Python

Sending the wrong number, type, or value of parameters to a function is another common cause for an exception. For example, a call to abs('hello') will raise a TypeError because the parameter is not numeric, and a call to abs(3, 5) will raise a TypeError because one parameter is expected. A ValueError is typically raised when the correct number and type of parameters are sent, but a value is illegitimate for the context of the function. For example, the int constructor accepts a string, as with int('137'), but a ValueError is raised if that string does not represent an integer, as with int('3.14') or int('hello').

Python's sequence types (e.g., list, tuple, and str) raise an IndexError when syntax such as data[k] is used with an integer k that is not a valid index for the given sequence (as described in Section 1.2.3). Sets and dictionaries raise a KeyError when an attempt is made to access a nonexistent element.

1.7.1 Raising an Exception

An exception is thrown by executing the **raise** statement, with an appropriate instance of an exception class as an argument that designates the problem. For example, if a function for computing a square root is sent a negative value as a parameter, it can raise an exception with the command:

```
raise ValueError('x cannot be negative')
```

This syntax raises a newly created instance of the ValueError class, with the error message serving as a parameter to the constructor. If this exception is not caught within the body of the function, the execution of the function immediately ceases and the exception is propagated to the calling context (and possibly beyond).

When checking the validity of parameters sent to a function, it is customary to first verify that a parameter is of an appropriate type, and then to verify that it has an appropriate value. For example, the sqrt function in Python's math library performs error-checking that might be implemented as follows:

```
def sqrt(x):
  if not isinstance(x, (int, float)):
    raise TypeError('x must be numeric')
  elif x < 0:
    raise ValueError('x cannot be negative')
  # do the real work here...
```

Checking the type of an object can be performed at run-time using the built-in function, isinstance. In simplest form, isinstance(obj, cls) returns True if object, obj, is an instance of class, cls, or any subclass of that type. In the above example, a more general form is used with a tuple of allowable types indicated with the second parameter. After confirming that the parameter is numeric, the function enforces an expectation that the number be nonnegative, raising a ValueError otherwise.

How much error-checking to perform within a function is a matter of debate. Checking the type and value of each parameter demands additional execution time and, if taken to an extreme, seems counter to the nature of Python. Consider the built-in sum function, which computes a sum of a collection of numbers. An implementation with rigorous error-checking might be written as follows:

```
def sum(values):
  if not isinstance(values, collections.Iterable):
    raise TypeError('parameter must be an iterable type')
  total = 0
  for v in values:
    if not isinstance(v, (int, float)):
      raise TypeError('elements must be numeric')
    total = total+ v
  return total
```

The abstract base class, collections.Iterable, includes all of Python's iterable containers types that guarantee support for the for-loop syntax (e.g., list, tuple, set); we discuss iterables in Section 1.8, and the use of modules, such as collections, in Section 1.11. Within the body of the for loop, each element is verified as numeric before being added to the total. A far more direct and clear implementation of this function can be written as follows:

```
def sum(values):
  total = 0
  for v in values:
    total = total + v
  return total
```

Interestingly, this simple implementation performs exactly like Python's built-in version of the function. Even without the explicit checks, appropriate exceptions are raised naturally by the code. In particular, if values is not an iterable type, the attempt to use the for-loop syntax raises a TypeError reporting that the object is not iterable. In the case when a user sends an iterable type that includes a nonnumerical element, such as sum([3.14, 'oops']), a TypeError is naturally raised by the evaluation of expression total + v. The error message

```
unsupported operand type(s) for +:  'float' and 'str'
```

should be sufficiently informative to the caller. Perhaps slightly less obvious is the error that results from sum(['alpha', 'beta']). It will technically report a failed attempt to add an int and str, due to the initial evaluation of total + 'alpha', when total has been initialized to 0.

In the remainder of this book, we tend to favor the simpler implementations in the interest of clean presentation, performing minimal error-checking in most situations.

1.7.2 Catching an Exception

There are several philosophies regarding how to cope with possible exceptional cases when writing code. For example, if a division x/y is to be computed, there is clear risk that a ZeroDivisionError will be raised when variable y has value 0. In an ideal situation, the logic of the program may dictate that y has a nonzero value, thereby removing the concern for error. However, for more complex code, or in a case where the value of y depends on some external input to the program, there remains some possibility of an error.

One philosophy for managing exceptional cases is to ***"look before you leap."*** The goal is to entirely avoid the possibility of an exception being raised through the use of a proactive conditional test. Revisiting our division example, we might avoid the offending situation by writing:

```
if y != 0:
  ratio = x / y
else:
  ... do something else ...
```

A second philosophy, often embraced by Python programmers, is that ***"it is easier to ask for forgiveness than it is to get permission."*** This quote is attributed to Grace Hopper, an early pioneer in computer science. The sentiment is that we need not spend extra execution time safeguarding against every possible exceptional case, as long as there is a mechanism for coping with a problem after it arises. In Python, this philosophy is implemented using a ***try-except*** control structure. Revising our first example, the division operation can be guarded as follows:

```
try:
  ratio = x / y
except ZeroDivisionError:
  ... do something else ...
```

In this structure, the "try" block is the primary code to be executed. Although it is a single command in this example, it can more generally be a larger block of indented code. Following the try-block are one or more "except" cases, each with an identified error type and an indented block of code that should be executed if the designated error is raised within the try-block.

The relative advantage of using a try-except structure is that the non-exceptional case runs efficiently, without extraneous checks for the exceptional condition. However, handling the exceptional case requires slightly more time when using a try-except structure than with a standard conditional statement. For this reason, the try-except clause is best used when there is reason to believe that the exceptional case is relatively unlikely, or when it is prohibitively expensive to proactively evaluate a condition to avoid the exception.

Exception handling is particularly useful when working with user input, or when reading from or writing to files, because such interactions are inherently less predictable. In Section 1.6.2, we suggest the syntax, fp = open('sample.txt'), for opening a file with read access. That command may raise an IOError for a variety of reasons, such as a non-existent file, or lack of sufficient privilege for opening a file. It is significantly easier to attempt the command and catch the resulting error than it is to accurately predict whether the command will succeed.

We continue by demonstrating a few other forms of the try-except syntax. Exceptions are objects that can be examined when caught. To do so, an identifier must be established with a syntax as follows:

```
try:
  fp = open('sample.txt')
except IOError as e:
  print('Unable to open the file:', e)
```

In this case, the name, e, denotes the instance of the exception that was thrown, and printing it causes a detailed error message to be displayed (e.g., "file not found").

A try-statement may handle more than one type of exception. For example, consider the following command from Section 1.6.1:

```
age = int(input('Enter your age in years: '))
```

This command could fail for a variety of reasons. The call to input will raise an EOFError if the console input fails. If the call to input completes successfully, the int constructor raises a ValueError if the user has not entered characters representing a valid integer. If we want to handle two or more types of errors in the same way, we can use a single except-statement, as in the following example:

```
age = -1                    # an initially invalid choice
while age <= 0:
  try:
    age = int(input('Enter your age in years: '))
    if age <= 0:
      print('Your age must be positive')
  except (ValueError, EOFError):
    print('Invalid response')
```

We use the tuple, (ValueError, EOFError), to designate the types of errors that we wish to catch with the except-clause. In this implementation, we catch either error, print a response, and continue with another pass of the enclosing while loop. We note that when an error is raised within the try-block, the remainder of that body is immediately skipped. In this example, if the exception arises within the call to input, or the subsequent call to the int constructor, the assignment to age never occurs, nor the message about needing a positive value. Because the value of age

will be unchanged, the while loop will continue. If we preferred to have the while loop continue without printing the `'Invalid response'` message, we could have written the exception-clause as

```
except (ValueError, EOFError):
  pass
```

The keyword, **pass**, is a statement that does nothing, yet it can serve syntactically as a body of a control structure. In this way, we quietly catch the exception, thereby allowing the surrounding while loop to continue.

In order to provide different responses to different types of errors, we may use two or more except-clauses as part of a try-structure. In our previous example, an EOFError suggests a more insurmountable error than simply an errant value being entered. In that case, we might wish to provide a more specific error message, or perhaps to allow the exception to interrupt the loop and be propagated to a higher context. We could implement such behavior as follows:

```
age = -1                        # an initially invalid choice
while age <= 0:
  try:
    age = int(input('Enter your age in years: '))
    if age <= 0:
      print('Your age must be positive')
  except ValueError:
    print('That is an invalid age specification')
  except EOFError:
    print('There was an unexpected error reading input.')
    raise                       # let's re-raise this exception
```

In this implementation, we have separate except-clauses for the ValueError and EOFError cases. The body of the clause for handling an EOFError relies on another technique in Python. It uses the raise statement without any subsequent argument, to re-raise the same exception that is currently being handled. This allows us to provide our own response to the exception, and then to interrupt the while loop and propagate the exception upward.

In closing, we note two additional features of try-except structures in Python. It is permissible to have a final except-clause without any identified error types, using syntax **except**:, to catch any other exceptions that occurred. However, this technique should be used sparingly, as it is difficult to suggest how to handle an error of an unknown type. A try-statement can have a **finally** clause, with a body of code that will always be executed in the standard or exceptional cases, even when an uncaught or re-raised exception occurs. That block is typically used for critical cleanup work, such as closing an open file.